
FOR ALL YOU NEEDS

SONDRA

USER GUIDE

Prepared by Anezatra

An open-source AI agent for all-purpose tasks

2026 Edition

Copyright (C) 2026, Agena Memory Systems

TABLE OF CONTENTS

English contents page for the full guide, generated from the UTF-8 usage source.

Sondra Usage Guide	4
What Sondra Does	4
Core Capabilities	4
Quick Start	5
Requirements	5
Install Poetry	5
Install Project Dependencies	5
Configure Your LLM	5
Audio Configuration	5
Check the CLI	6
Example Runs	6
OSINT Investigation	6
Deep Penetration Test	6
ADB Automation	6
General Assistant Mode	6
CLI Reference	6
Additional Runtime Features	7
Auto Mode Examples	7
Scan Modes	8
general	8
osint	8
adb	8
pentest	8
Scan Levels	8
quick	8
standard	8
deep	8
Custom Instructions	8
Configuration	9
Example Config File	10
Memory System	10
How Learning Works	10
How Recall Works	11
Important Memory Files and Folders	11
Memory Behavior Notes	11

Task Creation And Scheduling	12
Emotion Signals	12
Learning From Corrections And Mistakes	13
Interactive TUI Screen	14
Voice Speech	15
Language Support	15
SUBSYSTEM MENU Panel	16
Page 1: Activity Graph	16
Page 2: Agent Info	16
Page 3: System Resources	16
Page 4: Task Queue	16
Page 5: Memory Layers	16
Page 6: Emotion Weights	17
LLM Providers	17
Provider Notes	17
OpenAI	17
Anthropic	17
OpenRouter	17
Vertex AI	17
AWS Bedrock	18
Azure OpenAI	18
Local / Ollama	18
Tools	18
Browser	18
HTTP Proxy	18
Terminal	18
Python Runtime	18
File Editor	18
Screen Vision	18
ADB Bridge	18
Web Search	18
Todo and Planning	18
Thinking	18
Notes and Reporting	18
Sandbox Tooling	19
CI/CD Integration	19
Development Notes	20
Security and Responsibility	20
Final Thought	20

Sondra Usage Guide

Sondra is a unified agent platform built for people who do not want to manage a separate tool, script, or automation system for every task.

It brings general assistance, deep research, OSINT, penetration testing, ADB-based phone integration, intelligent memory, and adaptive interaction into a single cohesive environment.

This guide is the starting point for installation, configuration, scan modes, providers, tools, integrations, automation, memory behavior, and development notes.

What Sondra Does

Sondra is designed to provide a complete operational workspace for complex digital tasks.

Instead of forcing the user to switch between isolated agents, terminals, browsers, proxies, scripts, and research tools, Sondra combines them under one intelligent agent system.

Its purpose is simple:

One platform. Multiple domains. Less fragmentation. More execution.

Sondra can assist with:

- General autonomous tasks
- Deep research and investigation
- OSINT workflows
- Penetration testing
- Web application testing
- Android device automation through ADB
- Local and cloud LLM workflows
- Long-term memory and contextual understanding

What makes Sondra different is not only what it can do, but how it adapts.

Sondra can transform emotional, contextual, and behavioral signals into structured memory signals. This allows the system to understand the user more accurately over time and respond with better context, tone, and continuity.

It is not only a task runner.

It is an adaptive agent platform.

Core Capabilities

- Dynamic application testing with browser, proxy, terminal, Python, and notes.
 - Multi-agent orchestration for complex targets and parallel investigation.
 - OSINT, penetration testing, general assistance, and ADB phone integration.
 - Interactive TUI for live work and non-interactive mode for automation.
 - Support for local, cloud, and routed LLM providers through LiteLLM format.
-

- Intelligent memory system with semantic recall and profile-aware context.
- Emotional signal analysis for more adaptive and human-centered responses.
- Structured reporting, evidence collection, and reproducible task execution.

Quick Start

Requirements

- Python 3.12+
- Docker running locally
- Poetry or pipx
- An LLM model configured through environment variables

Install Poetry

```
curl -sSL https://install.python-poetry.org | python3 -
```

Install Project Dependencies

```
poetry install
```

Configure Your LLM

```
export SONDRA_LLM="openai/gpt-5"  
export LLM_API_KEY="your-api-key"
```

Audio Configuration

If you are running Sondra on WSL or a Windows environment, you need to manually configure ffmpeg for audio playback.

Example:

```
export FFPLAY_COMMAND_DIR="/mnt/c/ffmpeg/bin/ffplay.exe"
```

Make it persistent on WSL:

To ensure this setting is applied automatically in every session, add it to your `.bashrc` file:

```
echo 'export FFPLAY_COMMAND_DIR="/mnt/c/ffmpeg/bin/ffplay.exe"' >> ~/.bashrc
```

Then reload your configuration:

```
source ~/.bashrc
```

Linux systems:

On native Linux environments, installing FFmpeg is sufficient:

```
sudo apt install ffmpeg
```

Notes:

- ffplay is required for audio playback in Sondra.
- On Windows/WSL setups, make sure the path correctly points to your FFmpeg installation.
- You can verify the installation with:

```
ffplay -version
```

Check the CLI

```
poetry run sondra -h
```

First runs may pull a Docker sandbox image. Outputs are saved under run and log folders depending on mode and configuration.

Example Runs

OSINT Investigation

```
poetry run sondra -m osint -l standard --instruction "Investigate example.com"
```

Deep Penetration Test

```
poetry run sondra -m pentest -l deep --target https://example.com
```

ADB Automation

```
poetry run sondra -m adb -l standard --instruction-file ./adb_steps.txt
```

General Assistant Mode

```
poetry run sondra -m general
```

CLI Reference

Common options:

- -v, --version: Prints the installed Sondra version.

- `-m, --scan-mode`: Selects the operating mode. Supported values: `general`, `osint`, `adb`, `pentest`.
- `-l, --scan-level`: Selects the depth of execution. Supported values: `quick`, `standard`, `deep`.
- `-t, --target`: Defines the target. Can be a URL, domain, repository, local path, or IP address.
- `--instruction`: Provides inline task instructions.
- `--instruction-file`: Reads detailed instructions from a file.
- `-n, --non-interactive`: Runs without the interactive TUI.
- `--config`: Uses a custom JSON configuration file.
- `--subagents`: Forces subagent count where supported.
- `--retry`: Retries failed LLM requests before surfacing the final error.
- `--voice-speech`: Speaks assistant chat replies with local TTS.
- `-a, --auto`: Infers mode, level, target, and instruction from natural text.

Additional Runtime Features

Sondra includes several runtime options that are useful in daily work.

- Auto mode can turn a natural language request into mode, level, target, and instruction arguments.
- Retry mode can reconnect failed LLM requests when `--retry` is provided.
- Subagents can be forced with `--subagents` where the selected mode supports parallel work.
- Non-interactive mode can run Sondra in CI, scripts, and headless terminals.
- Instruction files can carry larger scopes, credentials, rules, and task plans.
- Voice speech can read assistant chat replies aloud when `--voice-speech` is enabled.
- The interactive TUI can show live agent state, traffic, task status, memory layers, and emotion weights.

Auto Mode Examples

Auto mode allows Sondra to infer what should happen from a natural language request.

```
poetry run sondra -a "Open https://example.com"
```

```
poetry run sondra -a "Analyze https://example.com carefully"
```

```
poetry run sondra -a "Run a penetration test against https://example.com"
```

Auto mode is useful when you want speed, but explicit mode selection is recommended for critical workflows.

Scan Modes

Use the mode that matches the job.

general

General autonomous assistant mode.

This mode is intended for ordinary assistant behavior, planning, contextual help, memory-aware responses, scheduled tasks, and long-term user continuity.

osint

Open-source intelligence and research mode.

This mode is intended for public information gathering, entity research, exposed asset discovery, reputation checks, and investigation workflows.

adb

Android device automation mode.

This mode is intended for instruction-driven phone control through ADB connectivity.

pentest

Penetration testing mode.

This mode keeps Strix-compatible security testing behavior while extending the overall platform with Sondra-specific architecture, memory, and interaction layers.

Scan Levels

quick

Fast execution for smoke tests, CI checks, and urgent tasks.

standard

Balanced execution for routine analysis and everyday workflows.

deep

Thorough exploration for difficult targets, release reviews, audits, and complex investigations.

Custom Instructions

Instructions tell Sondra what to focus on, what to avoid, what credentials to use, and what kind of result you expect.

Inline instructions are best for short tasks:

```
poetry run sondra -m osint -l standard --instruction "Focus on public mentions and leaked credential"
```


Instruction files are better for larger scopes:

```
poetry run sondra -m pentest -l deep --target https://app.example.com --instruction-file ./scope.md
```

A good instruction file should include:

- Credentials and roles
- In-scope and out-of-scope areas
- High-priority endpoints or features
- API keys, headers, or test accounts
- Known risks, assumptions, and limitations
- Reporting expectations
- Rules of engagement

Clear instructions produce better results.

Configuration

Sondra reads configuration from environment variables and optional config files.

Use `SONDRA_*` names for Sondra-specific settings.

Important variables:

- `SONDRA_LLM`: Required model name in LiteLLM format. Example: `openai/gpt-5`.
- `LLM_API_KEY`: Provider API key when needed.
- `LLM_API_BASE`: Custom endpoint for local or compatible APIs.
- `OPENAI_API_BASE`, `LITELLM_BASE_URL`, `OLLAMA_API_BASE`: Alternative base URLs.
- `LLM_TIMEOUT`: LLM request timeout.
- `SONDRA_REASONING_EFFORT`: Reasoning effort for supported reasoning models.
- `PERPLEXITY_API_KEY`: Enables web search and research tools.
- `SONDRA_MEMORY_DIR`: Memory database directory.
- `SONDRA_MEMORY_LOG_DIR`: Memory log directory.
- `SONDRA_MEMORY_EMBED_MODEL`: Embedding model for memory search.
- `SONDRA_MEMORY_EMBED_BASE_URL`: Embedding provider endpoint.
- `SONDRA_MEMORY_SYNC_MODE`: Memory sync behavior.
- `SONDRA_MEMORY_TEMPORAL_DECAY`: Enables memory importance decay.
- `SONDRA_IMAGE`: Docker sandbox image.
- `SONDRA_RUNTIME_BACKEND`: Runtime backend, normally `docker`.
- `SONDRA_SANDBOX_EXECUTION_TIMEOUT`: Tool execution timeout inside the sandbox.
- `SONDRA_SANDBOX_CONNECT_TIMEOUT`: Tool server connection timeout.
- `SONDRA_DISABLE_BROWSER`: Disables browser tools when set to `true`.

- `SONDRA_ADB_URL`: HTTP bridge URL for ADB mode.
 - `SONDRA_ADB_TIMEOUT`: ADB bridge request timeout.
 - `SONDRA_ADB_SCREENSHOT_PATH`: Path used for binary ADB screenshots.
-

Example Config File

```
{
  "env": {
    "SONDRA_LLM": "openai/gpt-5",
    "LLM_API_KEY": "sk-...",
    "SONDRA_REASONING_EFFORT": "high"
  }
}
```

Configuration files are useful when you want reproducible setups, shared team profiles, or clean separation between local and production environments.

Memory System

Sondra includes an intelligent memory system developed by Agena Memory Systems.

Memory is primarily used in general mode to help the assistant retain useful context across turns and future sessions.

It can store:

- Conversation history
- Semantic facts
- User profile data
- Active task states
- Emotional signals
- Scheduled tasks
- Long-running context

The goal is not to make the model permanently trained on the user.

The goal is to provide the right context at the right moment.

Memory is not model training.

Memory is structured context.

How Learning Works

Sondra can persist useful information from interaction history and organize it into structured memory.

Learning may include:

- Saving user and assistant turns as conversation messages.

- Extracting important user facts into semantic memory.
- Storing stable personal facts as profile facts.
- Saving recent task progress as task state.
- Tracking emotional signals when they are relevant to response quality.
- Storing scheduled tasks separately for dispatch.

This allows Sondra to keep continuity without forcing the user to repeat the same information again and again.

How Recall Works

Before generating a response, Sondra can build an automatic memory context.

This context may combine:

- Recent conversation lines
- Exact matches
- Keyword search
- Semantic search
- Profile facts
- Active task state
- Emotion-aware signals

Hybrid retrieval can use both lexical search and embeddings.

MMR ranking helps reduce repetition and noise.

Temporal decay can reduce the importance of older memories over time, allowing newer and more relevant information to become stronger in context.

Important Memory Files and Folders

- `SONDRA_MEMORY_DIR`: Where `memory.db` is stored.
 - `SONDRA_MEMORY_LOG_DIR`: Where memory logs and events are written.
 - `memory_events.jsonl`: Structured memory event log.
 - `memory_debug.log`: Debug-level memory activity log.
 - `config/memory_config.yaml`: Default memory configuration.
-

Memory Behavior Notes

- Scheduled tasks are intended for general mode only.
- Non-general modes such as `osint`, `adb`, and `pentest` do not use the task scheduling panel.
- Memory retrieval is prompt context, not model fine-tuning.
- Embedding-backed semantic search works best when the local embedding backend is reachable.

- Memory quality depends on clean extraction, good ranking, and useful context boundaries.
- Memory marker and signal detection is designed for Turkish and English only.
- Other languages can still be stored as raw conversation history, but semantic markers, task markers, and emotion markers are only expected to work reliably in Turkish and English.

Task Creation And Scheduling

Scheduled tasks are a general-mode memory feature.

When the user writes a time-based request, Sondra can detect the schedule, create a task record, store it in memory, and later dispatch it through a dedicated task agent.

Example requests:

```
saat 20:40 ta google.com'a gir
20 saniye sonra google.com'a gir
her 20 saniyede bir google.com'a gir
yarın saat 09:00'da raporu kontrol et
```

The task flow is:

1. The user writes a time-based instruction.
2. Sondra extracts the task text and schedule.
3. A task row is saved to the scheduled task store.
4. The TUI displays the task in the SUBSYSTEM MENU.
5. When the time arrives, Sondra creates a task subagent.
6. The task subagent executes the task and reports back to the root agent.
7. The root agent summarizes the result for the user.

Task labels in the panel:

- 20:40: A clock-based scheduled task.
- 00:20s: A task scheduled in seconds.
- 20:00m: A task scheduled in minutes.
- +: A recurring task.
- EXEC: A task is currently executing.
- CMPLT: A task recently completed.
- - - : - -: Empty slot or inactive task entry.

Task scheduling is not used by osint, adb, or pentest mode. In those modes the task panel shows that scheduled tasks are supported only in general mode.

Emotion Signals

Sondra can score emotional signals from user messages so replies can adapt tone and urgency. This is not a diagnosis and not a permanent personality profile. It is a lightweight interaction

signal used to make the assistant respond more appropriately.

Tracked emotion categories include:

- Happiness or positive tone.
- Sadness or low mood.
- Stress, frustration, or pressure.
- Anger or hostile wording.
- Neutral/default tone.
- Confidence score for how strong the detected signal is.

How emotion increases:

- Stronger words increase the matching emotion score.
- Repeated emotional wording can make the current signal stronger.
- Intensifiers such as "very", "really", "cok", or similar phrases can raise intensity.
- Direct phrases such as feeling bad, being frustrated, being happy, or being angry can push the relevant category upward.

How emotion weakens:

- Later messages without the same signal reduce the live emotional weight.
- Neutral or calmer messages move the current state back toward neutral.
- Softer wording produces lower confidence.
- The emotion panel compares recent/current scores, so arrows can go down when the latest signal is weaker than the previous one.

How Sondra uses it:

- If sadness or stress is detected, replies can become calmer and more supportive.
- If frustration is detected, replies can become more direct and practical.
- If neutral is dominant, replies stay normal.
- The model receives emotion context as prompt context; it is not retrained.

Learning From Corrections And Mistakes

Sondra can learn from interaction signals, but it does so through memory updates, not by changing the model weights.

When the user corrects the assistant, says something was wrong, or gives negative feedback, the system can treat that as a memory feedback signal. Useful memories can be reinforced, noisy memories can lose importance, and future retrieval can prefer more reliable context.

Examples of useful feedback:

```
hayir, boyle yapma  
bu yanlis oldu  
bir dahaki sefere daha kisa cevap ver  
bunu hatirla  
evet, bu dogru
```

What can happen internally:

- The correction is saved in conversation history.
- Relevant semantic memory can be updated or reinforced.
- Bad or noisy memory can be given lower importance.
- Future prompts can include the corrected preference or fact.
- Repeated successful recall can increase importance slightly.
- Old or unused semantic memory can decay over time.

This means Sondra can improve continuity and behavior across turns, but the underlying LLM is still the same model. Learning happens through stored context, ranking, feedback, and retrieval.

Interactive TUI Screen

The TUI is the live control screen shown during interactive runs.

Main areas:

- **TERMINAL:** The large left area. It shows boot output, user messages, assistant replies, tool activity, task messages, waiting state, and final results.
- **AGENT MAP:** The upper-right tree. It shows the root agent and any subagents currently created for the task.
- **SUBSYSTEM MENU:** The middle-right compact panel. It shows operational pages such as graph, agent info, resources, task queue, memory layers, and emotion.
- **Stats area:** Shows request/response rates, sent/received traffic, model, cost, Caído port, and version/status details.
- **Bottom input:** Where the user sends messages while the root agent is waiting.

Useful controls:

- **Ctrl + Q:** Request exit.
- **Esc:** Stop/interruption behavior where supported.
- **Up/Down:** Scroll terminal history.
- **Left/right arrows in SUBSYSTEM MENU:** Switch the subsystem page.
- **Center button in SUBSYSTEM MENU:** Stop or resume selected agent depending on current state.

The TUI is designed to make the system observable: you can see what the agent is doing, whether subagents exist, what mode is running, whether memory is active, and whether scheduled tasks are waiting or executing.

Voice Speech

Use `--voice-speech` to enable local speech output for assistant chat replies.

```
poetry run sondra -m general --voice-speech
```

What it speaks:

- Assistant chat replies.
- Natural responses after memory and task events.

What it does not speak:

- Raw tool output.
- Raw command output.
- Internal tool blocks.
- Empty control messages.

How playback works:

- Speech uses local Piper voice models.
- Turkish is the default voice path.
- English is used only when English is detected and an English model is available.
- On Linux, playback uses `ffplay` from `PATH`.
- On WSL or Windows-style setups, set `FFPLAY_COMMAND_DIR` to the `ffmpeg` binary directory.

Example WSL setup:

```
export FFPLAY_COMMAND_DIR="/mnt/c/ffmpeg/bin"
```

Controls:

- `+`: Increase voice volume in the TUI.
- `-`: Decrease voice volume in the TUI.

Language support:

- Speech supports Turkish and English only.
- Other languages fall back to Turkish-style speech handling.

Language Support

Sondra's language-aware memory and speech features are intentionally limited to Turkish and English.

- Memory signal support: Turkish and English.
- Speech/TTS support: Turkish and English.
- Marker-based learning, emotion detection, task phrasing, and speech language switching are not guaranteed outside Turkish and English.

- Non-TR/EN text may still be saved in conversation history, but advanced signal extraction should be treated as best effort only.

SUBSYSTEM MENU Panel

The SUBSYSTEM MENU is a compact multi-page status panel.

Use the < and > arrow buttons to move between pages.

Pages:

Page 1: Activity Graph

Shows compact request/response activity over time.

Use it to see whether the agent is actively sending requests, receiving replies, or staying idle.

Page 2: Agent Info

Shows the current model, mode, live agent count, progress state, and elapsed time.

Common fields:

- LLM: Current model family.
- MODE: Current mode such as GENERAL, OSINT, or ADB.
- AGNTS: Number of visible agents.
- PROGR: Running or stopped state.
- TIME: Elapsed runtime.

Page 3: System Resources

Shows CPU, RAM, disk, swap, and free memory indicators.

This helps identify whether slow behavior may be caused by system pressure.

Page 4: Task Queue

Shows scheduled tasks in general mode.

This is where entries like 20:40, 00:20s, 20:00m, EXEC, CMPLT, and + appear.

If the current mode is not general, scheduled tasks are not used and the panel shows that general mode is required.

Page 5: Memory Layers

Shows the current memory signal state.

Common fields:

- LAYERS: Number of active memory-related layers.
- SIGNAL: Latest memory signal type.
- ENTRY: Latest memory entry code.
- WEIGHT: Importance value.
- CONFIDENCE: Confidence value.

- REASON: Whether the memory signal is active.

This page helps confirm whether memory writes and memory updates are happening.

Page 6: Emotion Weights

Shows current emotion weights.

Common fields:

- Happiness/positive tone.
- Sadness.
- Stress/frustration.
- Neutral.
- Confidence/trend arrows.

Up arrows mean the latest signal is stronger than before. Down arrows mean it is weaker than before. Stable values mean no major change was detected.

LLM Providers

Sondra uses LiteLLM-style model names.

Examples:

```
openai/gpt-5  
anthropic/claude-sonnet-4-6  
openrouter/openai/gpt-5  
vertex_ai/gemini-3-pro-preview  
bedrock/anthropic.claude-4-5-sonnet-20251022-v1:0  
azure/your-deployment-name  
ollama/qwen3
```

Provider Notes

OpenAI

Use:

```
export SONDRA_LLM="openai/gpt-5"  
export LLM_API_KEY="your-api-key"
```

Anthropic

Use Claude model names and an Anthropic API key.

OpenRouter

Use OpenRouter model routing through LiteLLM format.

Vertex AI

Use Google Cloud Application Default Credentials.

AWS Bedrock

Use AWS credentials or IAM role based authentication.

Azure OpenAI

Use Azure deployment name, key, base URL, and API version.

Local / Ollama

Use local base URLs for private and local execution.

Local models are useful for privacy and experimentation, but smaller models may struggle with complex multi-agent workflows, long reasoning chains, and difficult security testing tasks.

Tools

Sondra agents operate with a practical execution toolkit.

Browser

Playwright-powered browser for navigation, forms, screenshots, and JavaScript-heavy flows.

HTTP Proxy

Caído-powered capture, replay, filtering, scope control, and request analysis.

Terminal

Persistent shell inside the sandbox for security tools, scripts, and system commands.

Python Runtime

Custom scripts for exploitation, parsing, validation, automation, and data processing.

File Editor

Read and modify source files when working on local repositories.

Screen Vision

Image and screenshot analysis for browser, ADB, and visual debugging workflows.

ADB Bridge

Instruction-driven Android control through a configured ADB bridge.

Web Search

Real-time research when PERPLEXITY_API_KEY is configured.

Todo and Planning

Internal task tracking for multi-step work, progress organization, and execution planning.

Thinking

Internal reasoning support for agents when a task needs structured analysis before tool execution.

Notes and Reporting

Track evidence, decisions, findings, and final reports.

Sandbox Tooling

The sandbox may include common recon, web, scanner, JavaScript, secret detection, auth, container, and proxy tooling.

Examples include:

- Nmap
- Nuclei
- SQLMap
- ffuf
- Katana
- Subfinder
- Semgrep
- Trivy
- TruffleHog
- jwt_tool
- Caido
- Playwright

The goal is not to replace every tool.

The goal is to allow agents to use the right tool at the right time.

CI/CD Integration

Use non-interactive mode for automation:

```
poetry run sondra -n -m pentest -l quick --target ./
```

Typical exit behavior:

- 0: Completed without vulnerabilities.
- 1: Execution error.
- 2: Vulnerabilities found in headless mode.

GitHub Actions, GitLab CI, Jenkins, and CircleCI workflows can follow the same basic structure:

1. Install Sondra.
2. Configure LLM secrets.
3. Run quick scans on pull requests.
4. Run deeper scans on schedules or release candidates.
5. Store reports as build artifacts.

Development Notes

Sondra is designed as an expandable agent platform.

It can be extended with:

- New modes
- New tools
- New providers
- New memory strategies
- New UI components
- New reporting formats
- New automation workflows

The architecture should remain modular.

Each subsystem should be powerful on its own, but cleanly integrated into the larger platform.

Security and Responsibility

Sondra can support security testing workflows, but it should only be used on systems where testing is authorized.

Always define:

- Scope
- Permission
- Target boundaries
- Testing windows
- Reporting expectations
- Data handling rules

Powerful automation requires responsible operation.

Final Thought

Sondra is built for users who want one intelligent system instead of dozens of disconnected tools.

It is not only a CLI.

It is not only a pentest wrapper.

It is not only a memory experiment.

It is a unified agent platform designed to think, act, remember, investigate, test, and adapt.

Less switching.
More execution.
One system.
Sondra.